1_data_extraction

2025-06-09

Introduction

This document will be responsible for extracting all of the raw data from the `long-beach-va.data`, `hungarian.data`, `switzerland.data`, and `cleveland.data` files containing the raw data sets and merging them into one data set. Once this has been accomplished Exploratory Data Analysis (EDA) will be performed on the entire merged data. Comprising **899 patient records** with **76 attributes**, including the target variable. The data is stored in an unconventional format: although it's a plain text file with space-separated values, **each patient record spans 10 rows**, and there are no header names provided in the data set specifically. The end of each record is indicated by the final field labeled `"name"`.

Importing Libraries

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ---------------------------------------- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(readr)
library(dplyr)
library(here)
```

```
## here() starts at /home/rstudio/Documents/ads503_project_g1
```

```
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
# set the column names per the heart-disease.names text file
full_cols <- c(
  "id",       # 1. patient ID
  "ccf",      # 2. ccf (social security #; now dummy)
  "age",      # 3. age in years
  "sex",      # 4. sex (1=male; 0=female)
  "painloc",  # 5. chest pain location (1=substernal; 0=otherwise)
  "painexer", # 6. provoked by exertion (1=yes; 0=no)
  "relrest",  # 7. relieved after rest (1=yes; 0=no)
  "pncaden",  # 8. sum of 5, 6, and 7
  "cp",       # 9. chest pain type (1-4)
  "trestbps", # 10. resting blood pressure (mm Hg)
  "htn",      # 11. history of hypertension (1=yes; 0=no)
  "chol",     # 12. serum cholesterol (mg/dL)
  "smoke",    # 13. is a smoker (1=yes; 0=no)
  "cigs",     # 14. cigarettes per day
  "years",    # 15. years as a smoker
  "fbs",      # 16. fasting blood sugar > 120 mg/dL (1=yes; 0=no)
  "dm",       # 17. diabetes history (1=yes; 0=no)
  "famhist",  # 18. family history of CAD (1=yes; 0=no)
  "restecg",  # 19. resting ECG results (0-2)
  "ekgmo",    # 20. month of exercise ECG reading
  "ekgday",   # 21. day of exercise ECG reading
  "ekgyr",    # 22. year of exercise ECG reading
  "dig",      # 23. digitalis used during ECG (1=yes; 0=no)
  "prop",     # 24. beta blocker used (1=yes; 0=no)
  "nitr",     # 25. nitrates used (1=yes; 0=no)
  "pro",      # 26. calcium channel blocker used (1=yes; 0=no)
  "diuretic", # 27. diuretic used during ECG (1=yes; 0=no)
  "proto",    # 28. exercise protocol (1=Bruce, 2=Kottus, … 12=arm ergometer)
  "thaldur",  # 29. duration of exercise test (minutes)
  "thaltime", # 30. time when ST depression was noted
  "met",      # 31. mets achieved
  "thalach",  # 32. maximum heart rate achieved
  "thalrest", # 33. resting heart rate
  "tpeakbps", # 34. peak exercise BP (first part)
  "tpeakbpd", # 35. peak exercise BP (second part)
  "dummy",    # 36. (unused)
  "trestbpd", # 37. resting blood pressure (duplicate)
  "exang",    # 38. exercise-induced angina (1=yes; 0=no)
  "xhypo",    # 39. (1=yes; 0=no)
  "oldpeak",  # 40. ST depression relative to rest
  "slope",    # 41. slope of the peak exercise ST segment (1-3)
  "rldv5",    # 42. height at rest
  "rldv5e",   # 43. height at peak exercise
  "ca",       # 44. number of major vessels (0-3) seen on fluoroscopy
  "restckm",  # 45. irrelevant
  "exerckm",  # 46. irrelevant
  "restef",   # 47. rest radionuclide ejection fraction
  "restwm",   # 48. rest wall motion abnormality (0-3)
  "exeref",   # 49. exercise radionuclide ejection fraction
  "exerwm",   # 50. exercise wall motion (0-3)
  "thal",     # 51. thalassemia (3=normal; 6=fixed; 7=reversible)
```

```
  "thalsev",  # 52. not used
  "thalpul",  # 53. not used
  "earlobe",  # 54. not used
  "cmo",      # 55. month of cardiac cath
  "cday",     # 56. day of cardiac cath
  "cyr",      # 57. year of cardiac cath
  "num",      # 58. diagnosis (0-4)
  "lmt",      # 59. left main coronary artery
  "ladprox",  # 60. left anterior descending proximal
  "laddist",  # 61. left anterior descending distal
  "diag",     # 62. diagonal
  "cxmain",   # 63. circumflex main
  "ramus",    # 64. ramus
  "om1",      # 65. obtuse marginal 1
  "om2",      # 66. obtuse marginal 2
  "rcaprox",  # 67. right coronary artery proximal
  "rcadist",  # 68. right coronary artery distal
  "lvx1",     # 69. not used
  "lvx2",     # 70. not used
  "lvx3",     # 71. not used
  "lvx4",     # 72. not used
  "lvf",      # 73. not used
  "cathef",   # 74. not used
  "junk",     # 75. not used
  "name"      # 76. last name (dummy string)
)
```

Data Importing

VA Long Beach

```
# using readLines, to read in each line of the long-beach-va.data file
# read_table, read_csv proved difficult to use to extract
# outline is to use the occurrence of 'name' to distinguish unique observations
# extract all the lines
va_raw_lines <- readLines(here("Data/Raw", "long-beach-va.data"))
# merge all lines into one string
va_all_text <- paste(va_raw_lines, collapse = " ")
# split each observation on name
va_obs <- strsplit(va_all_text, " name ")[[1]]

va_record_list <- map(va_obs, ~ {
  toks <- strsplit(.x, "\\s+")[[1]] # split strings at whitespace
  toks <- toks[toks != ""]  #empty token removal
  stopifnot(length(toks) == 75)
  data.frame(matrix(toks, nrow = 1),
             stringsAsFactors = FALSE)
})
# bind the rows
va_df_raw <- bind_rows(va_record_list)
# assign column names but leave out name
colnames(va_df_raw) <- full_cols[1:75]
# add name column and location column in
```

```r
va_df_raw$name <- "name"
va_df_raw$location <- "VA Long Beach"
```

Hungary

```r
lines <- readLines(here("Data/Raw", "hungarian.data"))

# Step 2: Combine every 10 lines per patient
patient_blocks <- split(lines, ceiling(seq_along(lines) / 10))

# Step 3: Parse each block into numeric + name
records <- map(patient_blocks, function(block) {
  tokens <- str_split(paste(block, collapse = " "), "\\s+")[[1]]
  name_value <- tail(tokens, 1)                      # last value is the name
  numeric_values <- as.numeric(tokens[-length(tokens)])
  c(numeric_values, name_value)                      # combine numeric + name
})

# Step 4: Combine into data frame
hu_df_raw <- do.call(rbind, records) %>%
  as.data.frame(stringsAsFactors = FALSE)
hu_df_raw <- hu_df_raw %>% slice(-295)    ## sliced out the blank row 295th
#dim(hu_df_raw)

# Step 5: Assign column names
colnames(hu_df_raw) <- full_cols

# Step 6: Replace -9 with NA
hu_df_raw <- hu_df_raw %>%
  mutate(across(where(is.character), ~ na_if(.x, "-9"))) %>%   # character cols
  mutate(across(where(is.numeric), ~ na_if(.x, -9))) %>%       # numeric columns
  mutate(location = "Hungary")

#ead(hu_df_raw)
```

Switzerland

```r
# Generative AI used to write code to read in .data file
# Step 1: Read the file
lines <- readLines(here("Data/Raw", "switzerland.data"))

# Step 2: Combine every 10 lines per patient
patient_blocks <- split(lines, ceiling(seq_along(lines) / 10))

# Step 3: Parse each block into numeric + name
records <- map(patient_blocks, function(block) {
  tokens <- str_split(paste(block, collapse = " "), "\\s+")[[1]]
  name_value <- tail(tokens, 1)                      # last value is the name
  numeric_values <- as.numeric(tokens[-length(tokens)])
  c(numeric_values, name_value)                      # combine numeric + name
})

# Step 4: Combine into data frame
```

```r
sw_df_raw <- do.call(rbind, records) %>%
  as.data.frame(stringsAsFactors = FALSE)

# Step 5: Assign column names
colnames(sw_df_raw) <- full_cols

# Step 6: Replace -9 with NA
sw_df_raw <- sw_df_raw %>%
  mutate(across(where(is.character), ~ na_if(.x, "-9"))) %>%   # character columns
  mutate(across(where(is.numeric), ~ na_if(.x, -9))) %>%       # numeric columns
  mutate(location = "switzerland")

#head(sw_df_raw)
```

Cleveland

```r
# using readLines, to read in each line of the cleveland.data file
# read_table, read_csv proved difficult to use to extract
# outline is to use the occurrence of 'name' to distinguish unique observations
# extract all the lines
clvd_raw_lines <- readLines(here("Data/Raw", "cleveland.data"),
                            warn = FALSE)
#clvd_raw_lines
# merge all lines into one string
clvd_all_text <- paste(clvd_raw_lines, collapse = " ")
# remove the corrupted tail end
clvd_all_text <- substr(clvd_all_text,
                        start = 1,
                        stop = 56933)
#clvd_all_text
# split each observation on name
clvd_obs <- strsplit(clvd_all_text, " name ")[[1]]
#clvd_obs
# map obs to record list
clvd_record_list <- map(clvd_obs, ~ {
  toks <- strsplit(.x, "\\s+")[[1]] # split strings at whitespace
  toks <- toks[toks != ""]  #empty token removal
  stopifnot(length(toks) == 75)
  data.frame(matrix(toks, nrow = 1),
             stringsAsFactors = FALSE)
})
# bind the rows
clvd_df_raw <- bind_rows(clvd_record_list)
# assign column names but leave out name
colnames(clvd_df_raw) <- full_cols[1:75]
# add name column and location column in
clvd_df_raw$name <- "name"
clvd_df_raw$location <- "Cleveland"
```

Data Merging

```r
# bind the rows and merge the df on top of each other
merged_df <- bind_rows(clvd_df_raw,
                       hu_df_raw,
                       sw_df_raw,
                       va_df_raw)
# merged_df

# write the merged raw file out to the raw data folder as 'merged_df_raw.csv'
# export to file
write.csv(merged_df, here("Data/Raw", "merged_df_raw.csv"),
          row.names = FALSE)
```